

Query Logic as a First-Class Architecture Concern

The Selector Pattern in Apex

FFLib Series · Session 004

John Storey

Staff Software Engineer · Thrivent



About me

Staff Software Engineer | Thrivent | Apex Enterprise Patterns maintainer

- **Apex Enterprise Patterns** — a maintainer of fflib, the library this series is built on
- **FinancialForce roots** — helped build fflib in the mid-2010s, alongside Andy Fawcett, John Daniel ("JD") and others

When I'm not working in the IT cloud, I'm flying above the real ones as a general aviation pilot.

thrivent®



FFLib Series

SESSION



Session #1 — Separation of Concerns in Apex: Why Your Future Self Will Thank You



Session #2 — Service Layers Explained: Coordinating Business Logic in Apex



Session #3 — Domain vs Service: Where Should Your Apex Logic Live?



Session #4 — Query Logic as a First-Class Architecture Concern

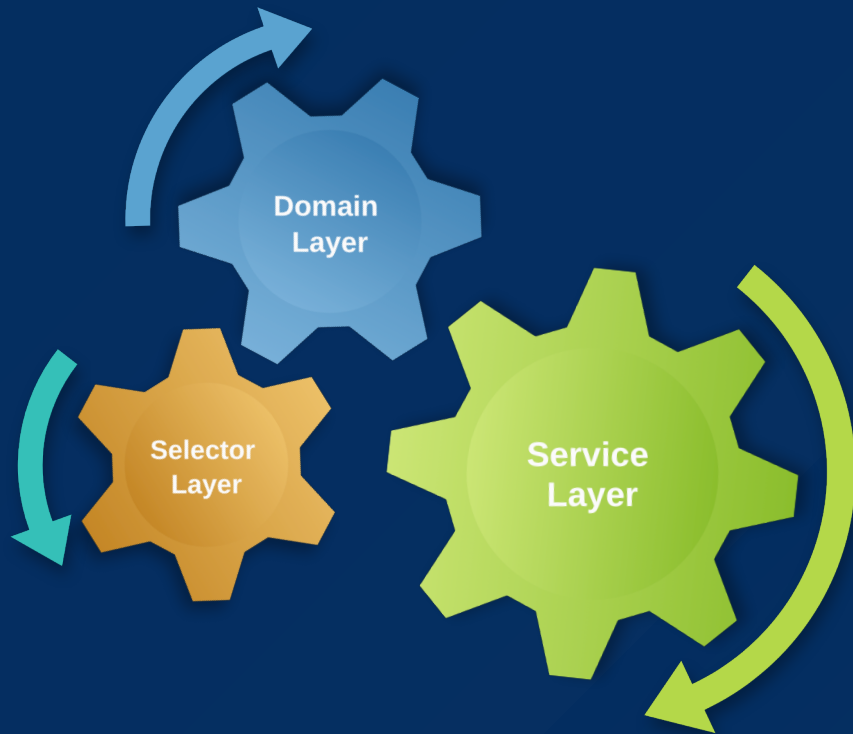


Session #5 — Mocking in Apex: Why It Changes Everything · Oct 20



Session #6 — Enterprise-Scale Apex Across Multiple Packages · Nov 3

Agenda



QUERY LOGIC

Recap



Why Query Logic Matters



Selector Principles



The fflib Selector



Warehouse App Selectors



Selector Evolution



Recap: bricks, a conductor, and who calls whom

CALLER	Service	Domain	Selector
Client (LWC, REST, Flow, Agent, Batch, ...)	•		•
Trigger Handler	•	•	•
Service	•	•	•
Domain		•	•
Selector			•

Demo: deploy fflib and the app

- ① Apex Mocks, ② Apex Common, ③ the Warehouse app (`force-app`) — the app needs both
- **A source deploy, not a package install** — each `sf project deploy start` runs from inside its own repo
- **Everything is in the session README** — github.com/CodeWithSally/CodeWithSally-Apex-English → Sessions/FFLibSeries/Session004...

■ ① Apex Mocks ■ ② Apex Common ■ ③ Warehouse app

① Apex Mocks

```
1 cd /tmp/fflib-apex-mocks
2 sf project deploy start --source-dir sfdx-source/apex-mocks \
3   --target-org session004-mfg --wait 15
```

② Apex Common

```
1 cd /tmp/fflib-apex-common
2 sf project deploy start --source-dir sfdx-source/apex-common \
3   --target-org session004-mfg --wait 20 --ignore-warnings
```

③ The Warehouse app

```
1 cd Sessions/FFLibSeries/Session004_QueryLogicAsFirstClassArchitectureConcern
2 sf project deploy start --source-dir force-app \
3   --target-org session004-mfg --wait 15 --ignore-errors
```

Good SOQL habits

- **No SOQL (or DML) inside loops**

-  `for (Id lineId : lineIds) { [SELECT ... WHERE Id = :lineId]; }`
-  One query with `WHERE Id IN :lineIds`, then work from a `Map`

- **Index loops, size cached** — about 2× faster than for-each: ≈500 ms against ≈1,000 ms of CPU over 200,000 integers, measured in our org
- **Bind variables**, never joined-in user input
- **Select only the fields you need; filter on indexed fields**
- **Bound your results** — `LIMIT`, a `QueryLocator`, or a cursor; mind the 50,000-row limit

What's wrong with this picture?

Developer "A" — the desk's LWC controller

WarehouseDeskController.cls

```
1 Map<Id, FulfillmentLine__c> pending =  
2   new Map<Id, FulfillmentLine__c>([  
3     SELECT Id, Name, ProductSku__c, Quantity__c  
4     FROM FulfillmentLine__c  
5     WHERE Status__c = 'Pending'  
6     AND FulfillmentOrder__r.Warehouse__c = :warehouseId  
7 ]]);
```

4

Developer "B" — the dispatch batch

DispatchLinesJob.cls

```
1 List<FulfillmentLine__c> pending = [  
2   SELECT Id, WeightKg__c, AssignedRobot__c  
3   FROM FulfillmentLine__c  
4   WHERE Status__c = 'Pending'  
5     AND AssignedRobot__c = null  
6     AND FulfillmentOrder__r.Warehouse__c IN :warehouseIds  
7     AND FulfillmentOrder__r.Status__c  
8       IN ('Released', 'InProgress')  
9   WITH SYSTEM_MODE  
10  ORDER BY Name  
11 ];
```

4

1

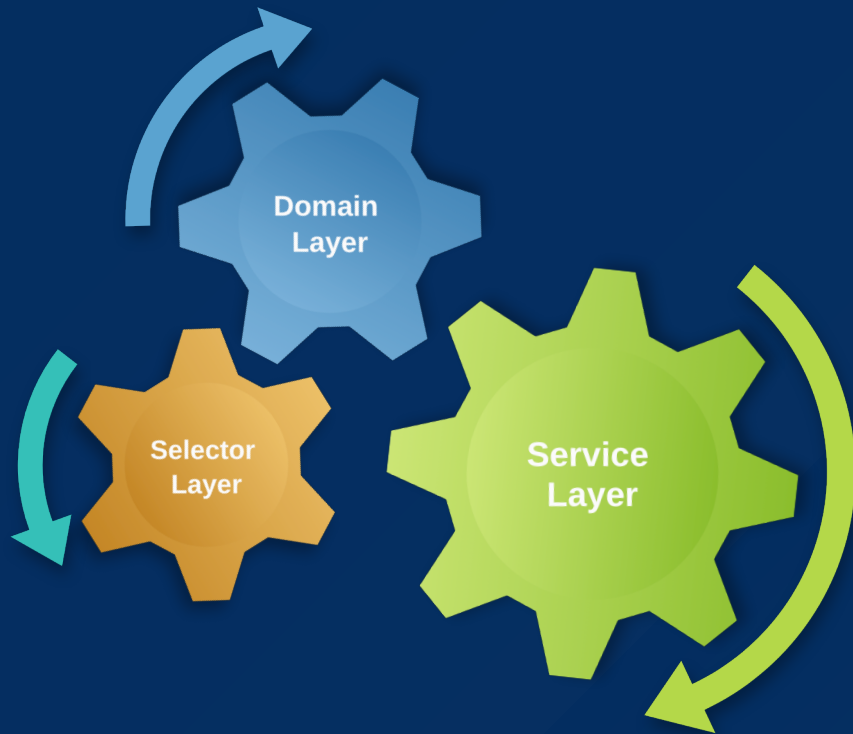
1

2

3

5

Agenda



QUERY LOGIC

Recap



Why Query Logic Matters



Selector Principles



The fflib Selector



Warehouse App Selectors



Selector Evolution



Where the pattern comes from

- **In a nutshell:** Selectors encapsulate query logic: one home for every question asked of the database
- **Data Mapper** (Fowler, *Patterns of Enterprise Application Architecture*) — moves data between objects and the database
- **Salesforce supplies the object half** — SObjects are the objects, so the Selector reduces to *where queries live*
- **Builder** — how fflib assembles those queries: `fflib_QueryFactory`
- **Selector** — Data Mapper's query half, built with a Builder

A query carries five concerns

- **Meaning** — the business rule a query encodes: which records qualify as *pending* or *idle*
- **Security** — who may see which records and fields
- **Performance** — selectivity, limits, large data volumes
- **Shape** — fields, order, relationships, aggregates, result types
- **Testability** — the seam where tests replace the database

Criteria: WHERE clause or Domain?

RobotsSelector.selectIdleByWarehouseWithModel

Robots.canAcceptWork

Selector – the coarse cut the database answers cheaply

```
1 fflib_QueryFactory query = newQueryFactory();
2 RobotModelsSelector.newInstance().configureQueryFactoryFields(query, 'Model__r');
3 return (List<Robot__c>) Database.query(
4     query.setCondition('Warehouse__c IN :warehouseIds AND Status__c = \'\'
5         + Robots.STATUS_IDLE + \'\'').toSQL()
6 );
```

Domain – the business rule that may change

```
1 Decimal wear = robot.WearPercent__c == null ? 0 : robot.WearPercent__c;
2 Decimal battery = robot.BatteryLevel__c == null ? 0 : robot.BatteryLevel__c;
3 return robot.Status__c == STATUS_IDLE
4     && robot.Health__c != HEALTH_CRITICAL
5     && wear < WEAR_CANNOT_WORK
6     && battery > MIN_BATTERY_TO_WORK;
```

i *Alternative: a formula field moves the rule into the WHERE clause, with its own baggage.*

The canonical Selector

- **One object, one Selector** — the default; most Selectors never go further
- **It alone defines its object's query knowledge:**
 - fields · default order · access mode · named conditions (what "open" or "idle" means)
- **Everything broader is built from these roots** — next slide
- **Aggregates belong here too** — coming up on slide 31
- Example: `WarehousesSelector`

Beyond one object: related records

- **Canonical:** one Selector per object — Warehouse
- **Composite Selectors:** usually methods on the object's own Selector, returning related records composed from their Selectors
- **Tight (master-detail):** Fulfillment Order $\rightleftarrows$ Fulfillment Lines — `...WithLines`, `...WithOrder`
- **Looser (lookups):** each object through its own Selector; the Service joins them — Warehouse $\rightarrow$ Robots; Robot $\rightarrow$ Maintenance Jobs
- **Small parent, always needed:** Robot $\rightarrow$ Robot Model — `...WithModel`
- **Feature Selector:** queries grouped by purpose when no single object owns them — e.g. campaign engagement over `CampaignMember`
- *Composite Selectors build on each object's Selector; they never redefine it.*

Selector Checklist

- **Method names say what is returned and how it's filtered:** `select...By...` , `count...` , `...AsQueryLocator` , `...AsCursor` , `...AsPaginationCursor`
 -  `selectIdleByWarehouseWithModel(warehouseIds)`  `getRobots(ids, true)`
- **Bulk in and out; empty in, empty out**
 -  `selectById(Set<Id> idSet)`  `selectById(Id robotId)`
- **Paramount fields by default, additions by need; consistent order**
- **User mode by default; aggregates and counts belong here too**

Questions you've already asked

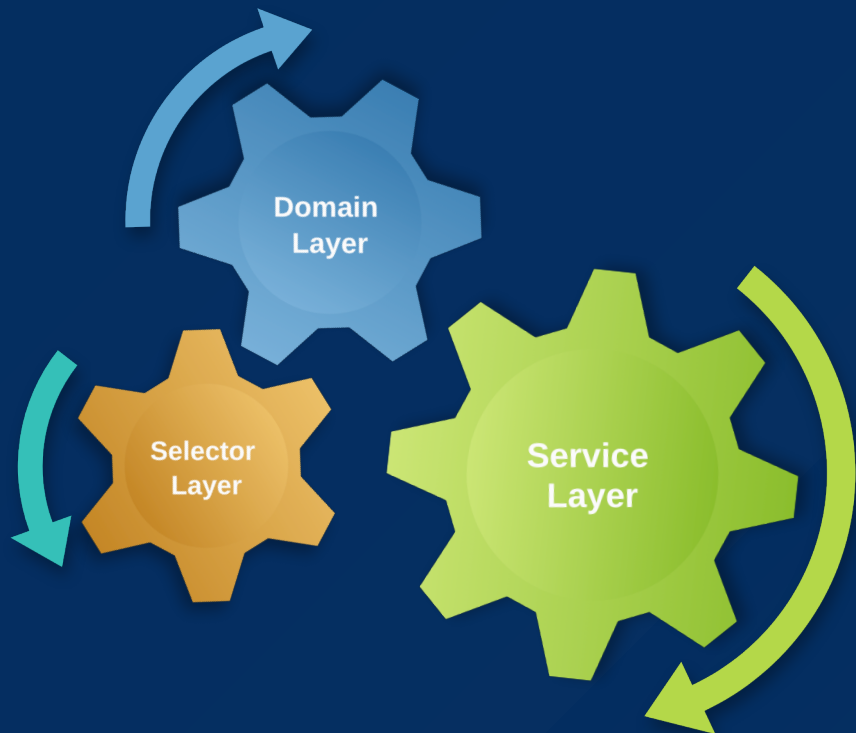
- **May a Domain call a Selector?** Yes — when a robot is reassigned, the Robots Domain checks that the target warehouse is Active, through `WarehousesSelector`
- **May a client call a Selector directly?** Yes, for a plain read — `DispatchWarehouse` calls `WarehousesSelector.selectByName`
- **One Selector per record type?** No — one per object, as a rule; say the record type in the method name. A Feature Selector groups queries by purpose when no single object owns them
- **Common fields or a new method?** Common fields go on the shared list; a special shape earns its own method, or a parameter that lets the consumer add fields

“

Questions so far?

”

Agenda



QUERY LOGIC

Recap



Why Query Logic Matters



Selector Principles



The fflib Selector



Warehouse App Selectors



Selector Evolution



Anatomy of an fflib Selector

WarehousesSelector.cls

```
1 public virtual inherited sharing class WarehousesSelector extends fflib_SObjectSelector {
2     public static WarehousesSelector newInstance() {           // composition
3         return new WarehousesSelector();
4     }
5     public WarehousesSelector() {                               // default posture
6         super(false, fflib_SObjectSelector.DataAccess.USER_MODE);
7     }
8     public List<Schema.SObjectField> getSObjectFieldList() {    // design: paramount fields
9         return new List<Schema.SObjectField>{
10             Warehouse__c.Id, Warehouse__c.Name, Warehouse__c.Status__c, Warehouse__c.Location__c
11         };
12     }
13     public Schema.SObjectType getSObjectType() {               // plumbing
14         return Warehouse__c.SObjectType;
15     }
16     public virtual List<Warehouse__c> selectActive() {         // a select method
17         return (List<Warehouse__c>) Database.query(
18             new QueryFactory().setCondition('Status__c = \'' + Warehouses.STATUS_ACTIVE + '\').toSOQL()
19         );
20     }
21 }
```

 Field tokens may help catch field-name issues before runtime.

Security: the defaults flipped

- **API 67.0+ (Summer '26):** Apex runs in **user mode** by default; no sharing keyword means **with sharing** — fflib's move is in the works
- **API 66.0 and earlier: system mode** by default
- **Defaults follow each class's API version** — the Warehouse app is 67.0; fflib is 63.0
- **So state the mode:** `.DataAccess.USER_MODE` appends `WITH USER_MODE` wherever the query executes

Data Access Layering

RobotsSelector.cls

```
1 // 1. Default: user mode – RobotsSelector.newInstance()
2 // 2. Posture chosen at instantiation
3 public RobotsSelector(fflib_SObjectSelector.DataAccess access) {
4     super(false, access);
5 }
6 // 3. Method argument: this query only; the instance is unchanged
7 public virtual List<Robot__c> selectById(Set<Id> idSet, fflib_SObjectSelector.DataAccess access) {
8     return (List<Robot__c>) Database.query(
9         newQueryFactory(access).setCondition(Robot__c.Id + ' IN :idSet').toSOQL()
10    );
11 }
```

- Entry points still declare
- Consumers stay inherited
- Queries declare user mode
- Elevation is named, never ambient

What the base class gives you

- **Required:** `getSObjectType` , `getSObjectFieldList`
- **Free queries:** `selectSObjectsById` , `queryLocatorById`
- `newQueryFactory()` — creates a query factory pre-loaded with fields, order, and mode
- **Composition:**
 - `configureQueryFactoryFields` — adds a parent Selector's fields under a relationship path
 - `addQueryFactorySubselect` — adds a child Selector's fields and order as a subquery
- **Field sets:** `includeFieldSetFields` — adds the constituent fields of a field set
- **Default order:** the name field (unless encrypted) → CreatedDate → Id

The Builder in action

■ FulfillmentLinesSelector.getOrderBy

■ WarehousesSelector.selectByName

Order is a business rule, set once

```
1 public override String getOrderBy() {  
2     return 'FulfillmentOrder__r.Priority__c, Name';  
3 }
```

A method states only its condition

```
1 Map<String, Warehouse__c> warehouseByName = new Map<String, Warehouse__c>();  
2 if (names == null || names.isEmpty()) {  
3     return warehouseByName;  
4 }  
5 for (Warehouse__c warehouse : (List<Warehouse__c>) Database.query(  
6     newQueryFactory().setCondition('Name IN :names').toSOQL()  
7 )) {  
8     warehouseByName.put(warehouse.Name, warehouse);  
9 }  
10 return warehouseByName;
```

 *An index loop may help with larger result sets.*

Paramount fields, plus what you need

RobotsSelector.cls

```
1 // In-method: a field for this method's purpose
2 public virtual List<Robot__c> selectByIdWithOwner(Set<Id> idSet) {
3     return (List<Robot__c>) Database.query(
4         newQueryFactory().selectField(Robot__c.OwnerId).setCondition(Robot__c.Id + ' IN :idSet').toSOQL(
5     );
6 }
7 // Consumer-supplied: fields of particular interest
8 public virtual List<Robot__c> selectById(Set<Id> idSet, Set<Schema.SObjectField> additionalFields) {
9     fflib_QueryFactory query = newQueryFactory();
10    if (additionalFields != null) {
11        query.selectFields(additionalFields);
12    }
13    return (List<Robot__c>) Database.query(query.setCondition(Robot__c.Id + ' IN :idSet').toSOQL());
14 }
15 // Declarative: field sets via includeFieldSetFields (the hook AT4DX builds on)
16
```

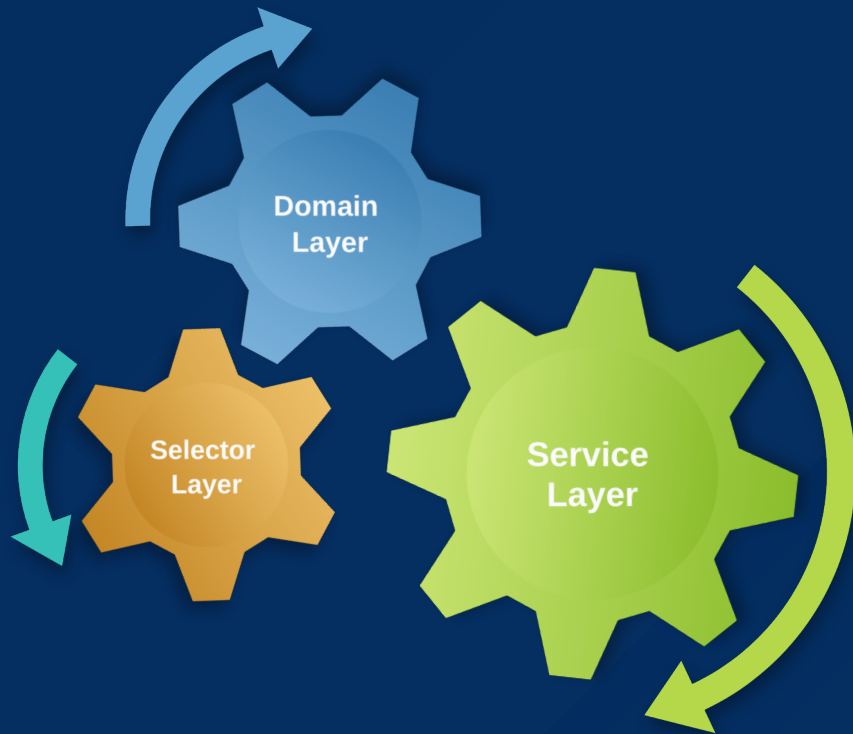
Tokens, not strings: shift left. Nobody removes the core.

“

Questions before the Warehouse app?

”

Agenda



QUERY LOGIC

Recap



Why Query Logic Matters



Selector Principles



The fflib Selector



Warehouse App Selectors



Selector Evolution



Tight relationships, in code

■ FulfillmentOrdersSelector.selectByIdWithLines

■ FulfillmentLinesSelector.selectPendingUnassignedByWarehouseWithOrder

Children through a subselect

```
1 fflib_QueryFactory ordersQuery = newQueryFactory();
2 FulfillmentLinesSelector.newInstance().addQueryFactorySubselect(ordersQuery);
3 return (List<FulfillmentOrder__c>) Database.query(
4     ordersQuery.setCondition('Id IN :idSet').toSOQL()
5 );
```

Parent fields borrowed from the parent's Selector

```
1 fflib_QueryFactory query = newQueryFactory();
2 FulfillmentOrdersSelector.newInstance().configureQueryFactoryFields(query, 'FulfillmentOrder__r');
3 return (List<FulfillmentLine__c>) Database.query(
4     query.setCondition(
5         'Status__c = \'' + FulfillmentLines.STATUS_PENDING + '\''
6         + ' AND AssignedRobot__c = null'
7         + ' AND FulfillmentOrder__r.Warehouse__c IN :warehouseIds'
8         + ' AND FulfillmentOrder__r.Status__c IN (\''
9         + FulfillmentOrders.STATUS_RELEASED + '\', \''
10        + FulfillmentOrders.STATUS_IN_PROGRESS + '\')'
11     ).toSOQL()
12 );
```

Who wrote this query?

■ FulfillmentLinesSelector.selectPendingUnassignedByWarehouseWithOrder(...) → toSOQL()

```
1 SELECT id, name, fulfillmentorder__c, status__c,
2   productsku__c, quantity__c, weightkg__c, assignedrobot__c,
3   fulfillmentorder__r.id, fulfillmentorder__r.name,
4   fulfillmentorder__r.warehouse__c, fulfillmentorder__r.status__c,
5   fulfillmentorder__r.priority__c, fulfillmentorder__r.duedate__c
6 FROM FulfillmentLine__c
7 WHERE Status__c = 'Pending' AND AssignedRobot__c = null
8    AND FulfillmentOrder__r.Warehouse__c IN :warehouseIds
9    AND FulfillmentOrder__r.Status__c IN ('Released', 'In Progress')
10 WITH USER_MODE
11 ORDER BY FulfillmentOrder__r.Priority__c ASC NULLS FIRST , Name ASC NULLS FIRST
```

One statement, three sources of truth, nothing retyped.

Looser relationships, in code

■ DispatchService.dispatchWarehouses

```
1 List<FulfillmentLine__c> pending =  
2     lineSelector.selectPendingUnassignedByWarehouseWithOrder(warehouseIds);  
3 if (pending.isEmpty()) {  
4     return;  
5 }  
6 List<Robot__c> idle = robotSelector.selectIdleByWarehouseWithModel(warehouseIds);
```

Return shapes

SHAPE

WHEN

List or Map

Everyday reads

AggregateResult → Map

Counts and summaries

QueryLocator

Batch Apex

Cursor

Chunked async processing

PaginationCursor

UI paging

Aggregates

RobotsSelector.countAvailableByWarehouse

```
1 Map<Id, Integer> counts = new Map<Id, Integer>();
2 for (AggregateResult row : [
3     SELECT Warehouse__c warehouseId, COUNT(Id) robotCount
4     FROM Robot__c
5     WHERE Warehouse__c IN :warehouseIds
6         AND Status__c = :Robots.STATUS_IDLE
7         AND (Health__c = null OR Health__c != :Robots.HEALTH_CRITICAL)
8         AND (WearPercent__c = null OR WearPercent__c < :Robots.WEAR_CANNOT_WORK)
9         AND BatteryLevel__c > :Robots.MIN_BATTERY_TO_WORK
10    WITH USER_MODE
11    GROUP BY Warehouse__c
12 ]) {
13     counts.put((Id) row.get('warehouseId'), (Integer) row.get('robotCount'));
14 }
15 return counts;
```

Batch versus Queueable chunking

■ WarehousesSelector.selectActiveAsQueryLocator

■ FulfillmentLinesSelector.selectPendingByWarehouseAsCursor

QueryLocator → Batch Apex

```
1 return Database.getQueryLocator(  
2     newQueryFactory().setCondition('Status__c = \'' + Warehouses.STATUS_ACTIVE + '\').toSOQL()  
3 );
```

Cursor → chunked Queueables

```
1 String orderRelationship = FulfillmentLine__c.FulfillmentOrder__c  
2     .getDescribe().getRelationshipName();  
3 return Database.getCursor(  
4     newQueryFactory()  
5         .setCondition(  
6             FulfillmentLine__c.Status__c + ' = \'' + FulfillmentLines.STATUS_PENDING + '\'  
7             + ' AND ' + orderRelationship + '.' + FulfillmentOrder__c.Warehouse__c  
8             + ' IN :warehouseIds'  
9         )  
10    .toSOQL()  
11 );
```

UI paging

RobotsSelector.selectByWarehouseAsPaginationCursor

```
1 return Database.getPaginationCursor(  
2     newQueryFactory()  
3         .setCondition(Robot__c.Warehouse__c + ' IN :warehouseIds')  
4         // Name isn't unique, so Id breaks ties  
5         .setOrdering(Robot__c.Name, fflib_QueryFactory.SortOrder.ASCENDING)  
6         .addOrdering(Robot__c.Id, fflib_QueryFactory.SortOrder.ASCENDING)  
7         .toSOQL()  
8 );
```

Live: without-paging.apex , then with-paging.apex · up to 100K rows · credit Andy's Jan 2026 post

```
public virtual void dispatchWarehouses(Set<Id> warehouseIds) {
```

1. **Selector** — pending, unassigned lines at these warehouses, with their orders
2. **Selector** — idle robots at these warehouses, with their models
3. **Domain** — keep robots that can legally accept work
4. **Service** — match one robot per line, within one warehouse
5. **Domain** — start robots, assign lines, mark orders in progress
6. **Service** — one Unit of Work commits

```
}
```

The mocking seam

■ DispatchService.newInstance ■ DispatchServiceTest.cls

Composition: real Selectors by default

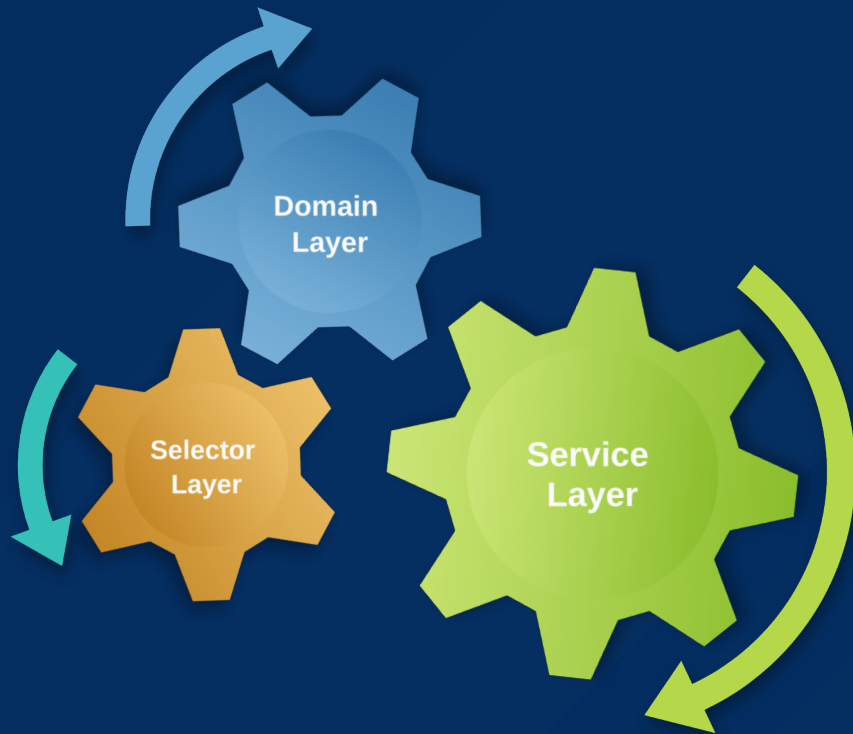
```
1 public static DispatchService newInstance() {  
2     return new DispatchService(  
3         FulfillmentLinesSelector.newInstance(),  
4         RobotsSelector.newInstance(),  
5         FulfillmentOrdersSelector.newInstance()  
6     );  
7 }
```

Tests: stub the virtual methods, inject through the constructor

```
1 RobotsSelector robotSelectorMock = (RobotsSelector) mocks.mock(RobotsSelector.class);  
2 mocks.startStubbing();  
3 mocks.when(robotSelectorMock.selectIdleByWarehouseWithModel(warehouseIds))  
4     .thenReturn(new List<Robot__c>{ robot });  
5 mocks.stopStubbing();  
6 new DispatchService(lineSelectorMock, robotSelectorMock, orderSelectorMock)  
7     .dispatchWarehouses(warehouseIds);
```

① **Session #5 — Mocking in Apex** · Oct 20: the deep dive

Agenda



QUERY LOGIC

Recap



Why Query Logic Matters



Selector Principles



The fflib Selector



Warehouse App Selectors



Selector Evolution



Selector Evolution

- **Explicit access modes**

-  `DataAccess.USER_MODE` , and the API 67.0 user-mode default
-  `enforceFLS()` , CRUD/FLS constructor flags, `without sharing` inner classes

- **Fields**

-  Common fields, with additions by need
-  Kitchen-sink Selectors that fetch everything for everyone

- **Composition**

-  Instances composed through `newInstance()` and constructors; feature Selectors where queries belong together
-  Static query methods

- **New shapes:** cursors, absorbed without changing the pattern

The library keeps moving

- `setAllRows(Boolean)` **overload** — Reinier van den Assum, Sep 2026
- `@NamespaceAccessible` — Aug 2026
- **Apex Mocks** — no longer a dependency
- **AT4DX** — cross-package extension through field sets (`includeFieldSetFields`)

Go deeper

- **Matt Gerry** — chapters 12–14 at apex-enterprise-patterns.dev; moving to the AEP GitHub soon
- **fflib-apex-common** — github.com/apex-enterprise-patterns/fflib-apex-common
- **The Warehouse sample** — this session's repo and README
- **Andy Fawcett** — andyinthecloud.com
 - [Recent updates and thoughts on the Application class](#) (Apr 2026)
 - [What does the new Apex User Mode Default mean for you?](#) (Apr 2026)
 - [Infinite scrolling with Apex Pagination Cursors](#) (Jan 2026)

“ *Ask the database a question once, in one place, in words the business would recognise.* ”

What's Next — FFLib Series

SESSION



Session #1 — Separation of Concerns in Apex: Why Your Future Self Will Thank You



Session #2 — Service Layers Explained: Coordinating Business Logic in Apex



Session #3 — Domain vs Service: Where Should Your Apex Logic Live?



Session #4 — Query Logic as a First-Class Architecture Concern



Session #5 — Mocking in Apex: Why It Changes Everything · Oct 20



Session #6 — Enterprise-Scale Apex Across Multiple Packages · Nov 3

Thank you!

John Storey · Code With Sally
FFLib Series · Session 004

